

Bank-Isolated KV Placement for LLM Decode

Where a paged KV cache lands in DRAM, and what that placement costs the memory system

ONNXim + Ramulator2 · HBM3 @ 6.4 Gbps, 16 ch × 2 pch × 4 bg × 4 bank
1 KB row · 32 B request · 4 cores × 128×128 systolic, fp16
LLaMA-2 7B, one transformer block, decode · 128 Azure production requests

ABSTRACT

A decoding step presents the memory system with two streams of very different character. The weight stream is read once per step for the entire batch and is fixed in size, 405 MB for one LLaMA-2 7B block whether the batch holds one request or a hundred. The KV cache stream is read once per token resident in the batch, and its addresses are produced by a paging allocator rather than by a static layout. On a batch of 128 production requests the KV stream is **90% of DRAM traffic**, which makes the physical placement of the KV cache the dominant influence on how the memory system behaves.

We characterise KV placement on a cycle-level model of an NPU and an HBM3 controller, separating the weight, KV and activation streams at the controller so each is measured on its own. The layout paged-attention systems use — sixteen-token pages holding every head, with each head's slice 4 KB long — achieves a **79.4%** row-buffer hit rate and runs the batch in 24.13M DRAM cycles. A layout that keeps each head's context contiguous achieves **70.0%**, the worst hit rate we measure, and runs the same batch **1.36× faster**. Row-buffer hit rate therefore does not order these layouts by speed. The reason is that the higher hit rate is produced by *concentration*: four cores working adjacent heads share a row, and four adjacent heads occupy exactly one bank, so sharing a row means sharing a bank. Counting distinct banks a channel touches per 2,000-cycle window, the concentrated layout reaches a median of 8 of 32 against 27 for the contiguous one, and with a quarter of the banks carrying the traffic a row switch has no other open row to hide behind.

The two layouts are endpoints of a single axis, the number of bytes of one head that stay contiguous, and the axis has an interior point neither of them occupies. Choosing that unit to be exactly one DRAM row-stripe places the head index on the bank-select bits, giving each head a private bank: **91.9%** KV row hit, 47% fewer activations than the block-major layout and 61% fewer than the contiguous one, for identical bytes moved. Its cost is that a bank-confined stream cannot overlap its own row switch, so it runs at about 55% of a bank's duty; two banks per head recovers most of the bandwidth while keeping most of the locality.

Two limits are worth stating with the result. Raising the number of concurrently streaming heads — by adding cores, or by interleaving a core's outstanding loads — reliably buys bandwidth and reliably costs locality, because every mechanism we tried for it also shortens the run a stream pulls from each row; no configuration measured reaches 90% on both. And the placement assumes a linear address mapping in which the bank index is a contiguous field; a controller that hashes other address bits into that field would scramble the head-to-bank assignment, so the timing ratios are conditional on the mapper used.

CONTENTS

PART I — WHAT DECODE ASKS OF DRAM

- 1 Two streams, and why only one can be placed
- 2 What a row visit costs

PART II — PLACEMENT

- 3 One axis: bytes of a head kept contiguous
- 4 Putting the head index on the bank field

PART III — METHOD

- 5 Model, workload, and what is measured

PART IV — RESULTS

- 6 The production batch
- 7 Concurrency, and the isolation dial
- 8 What holds, and what is not settled

PART I

What decode asks of DRAM

1 Two streams, and why only one can be placed

Weights are read once per step for the whole batch. The KV cache is read once per token in the batch. Only the second is placed by software.

A decoding step runs the batch through the model's linear layers, then computes attention for each request against its own cached keys and values. The systolic array is weight-stationary: it holds one 128×128 tile of a weight matrix and streams every request's activations through that tile before replacing it. A weight byte therefore crosses the DRAM boundary **exactly once per step**, and the weight stream is 405 MB for one transformer block regardless of batch size — measured identically at batch 1, 8 and 128. Batching buys activation reuse inside the array, not weight reuse in DRAM.

The KV cache behaves in the opposite way. Each request owns its cache, no part of it is shared, and every token of every request is read once per step. One token of one head is $\text{head_dim} \times \text{precision} = 256$ B of keys and the same of values, so a batch holding 200,064 tokens reads **3.3 GB of KV against 405 MB of weights** in a single step. Weight traffic is a fixed cost amortised over the batch; KV traffic scales with it. That asymmetry is the economic argument for continuous batching, and it is also why placement matters for one stream and not the other.

The addresses differ in kind as well as in volume. A weight tensor is laid out once at load time and its address list is identical on every step. A KV cache is *paged*: it is allocated in fixed-size blocks from a pool, and a request's blocks are wherever the allocator had space when the request arrived. The block table is the one part of the address stream that cannot be derived from configuration, which is why the measurements here drive a serving simulation rather than a synthetic pattern.

Within a block, though, the layout is a free choice, and it is the choice this report is about. A block holds B tokens for *every* head, and how those heads are arranged inside it decides which addresses are adjacent — which in turn decides the longest contiguous run any traversal can produce. Nothing downstream lengthens a run. A scheduler can reorder requests but cannot re-neighbour them, and a memory controller can only exploit locality that placement already created.

The phases are also separated in time, which matters when reading the measurements. In the linear layers the DRAM sees weights and activations finely interleaved, alternating at the granularity of individual requests. During attention it sees keys and values only, one request at a time, with that request's heads spread across the cores. On the production batch the attention phase is 96% of the step.

2 What a row visit costs

A bank has one row buffer, and the cost of missing it depends on which way it is missed.

A DRAM bank cannot read a cell directly. To read anything it first *activates* a row, copying roughly a kilobyte of cells into the bank's sense amplifiers, which takes trcd — 23 cycles in this configuration — during which that bank delivers nothing. Column reads then stream out of the open row cheaply, back to back. Before a different row can be activated the current one must be written back and the bitlines restored, another trp of

23 cycles. The row buffer is a cache with exactly one line per bank, and in this configuration a channel has thirty-two of them.

A request that does not hit the open row fails in one of two ways, and the two do not cost the same. If the bank is *closed*, the request pays one activate, and the precharge that closed the row was issued earlier, off the critical path. If the bank is *open on the wrong row*, the request pays a precharge and an activate back to back, with that bank's queue stalled behind it. A conflict is therefore roughly twice a miss, and the controller's precharge audit confirms the distinction is physical rather than bookkeeping: across a full run no precharge is ever left unfollowed by an activate, so a miss is not a wasted close but a close whose cost was successfully hidden.

This gives the two metrics used throughout. **Reads per activation** measures how much of a row visit's fixed cost is amortised; a 1 KB row holds thirty-two 32 B requests, so thirty-two is a full row and anything less is waste. **While-busy bandwidth** measures how much data the bus carries during cycles when requests are actually outstanding, which separates the memory system's efficiency from the NPU's ability to keep it fed.

One more quantity matters for what follows, and it is the one a hit rate cannot see. A bank in the middle of a row switch delivers nothing for $t_{RP} + t_{RCD}$, so whether the data bus stays busy depends on how many *other* banks hold open rows with work queued against them. A channel here has thirty-two banks; a placement that spreads its concurrent streams across many of them hides every row switch behind another bank's data, and a placement that concentrates them into a few has nothing to hide behind. We report this directly as **banks active**, the number of distinct banks a channel touches in a 2,000-cycle window.

Table 1 The three outcomes and what each costs on the bank that serves it. Cycles are for the HBM3 timing set used here.

OUTCOME	BANK STATE ON ARRIVAL	COMMANDS	APPROX. COST
hit	open on the requested row	CAS	data only
miss	closed	ACT + CAS	~23 cycles
conflict	open on a different row	PRE + ACT + CAS	~46 cycles, queue stalled

A hit rate collapses the second and third rows into one number. Two placements with the same hit rate can differ by a factor of two in what those non-hits cost, and §6 shows two that do.

PART II

Placement

3 One axis: bytes of a head kept contiguous

The placements are not alternatives. They are one formula evaluated at different points.

Write the address of a KV element as a function of its head index and of the position that element occupies within that head's own stream. Every placement considered here, including block-major paging, is the same flattening:

$$\text{addr}(\text{head}, p) = \underbrace{(p / U) \times (NH \times U)}_{[- \text{outer} -]} + \underbrace{\text{head} \times U}_{[\text{head}]} + \underbrace{(p \bmod U)}_{[\text{inner}]}$$

with NH the number of KV heads and **U the number of bytes of one head that stay contiguous** before the layout moves to the next head. Cut a head's stream into pieces of U bytes; piece c occupies slot head of the c -th group of NH pieces. One parameter, and the placements fall out of it.

Because one token of one head is 256 B, U is the page block size measured in bytes. A sixteen-token block — the granularity paged-attention implementations commonly use — is **4 KB of one head**, after which the layout jumps to the next head and the current head's data resumes 128 KB later. We call this *block-major*. At the other extreme, giving each head its entire context contiguously is the limit $U \rightarrow \infty$: $p < U$ always, the outer term vanishes, and the expression reduces to $\text{head} \times \text{context} + p$. We call this *head-major*.

Table 2 The axis. A row-stripe is 16 KB in this configuration: one 1 KB row in each of the sixteen channels (§4). Cycles are the production batch, four cores.

U	BLOCK SIZE	PLACEMENT	ROW-STRIPES PER HEAD	CYCLES
4 KB	16	block-major, 16-token pages	$\frac{1}{4}$ — four heads share a row	24.13M
8 KB	32	—	$\frac{1}{2}$	24.16M
16 KB	64	bank-isolated, $K = 1$	1 — one head per row	23.22M
32 KB	128	bank-isolated, $K = 2$	2	18.88M
64 KB	256	$\approx$ head-major	4	17.81M
context	—	head-major	whole context	17.68M

Two consequences are worth stating before the mechanism. First, the interior points of this axis are reachable **through the page-size parameter a paged KV cache already exposes**. Block 64 and the bank-isolated placement of §4 are the same addresses arrived at by two different arguments, and they measure 23,223,415 and 23,214,626 cycles, four hundredths of a percent apart on a model that is bit-deterministic. A sixteen-token page sits at the worst point on the axis for row locality.

Second, the equivalence is specific to a model and a precision, since U is $B \times \text{head_dim} \times \text{precision}$: at int8 the same 16 KB is 128 tokens, and a model with 64-dimensional heads would need twice the block

size for the same effect. *One row-stripe per head* is the portable statement of the result; *block 64* is what it happens to mean here.

4 Putting the head index on the bank field

Choose U to be exactly one row-stripe and the layout formula stops being arithmetic: it becomes a bit-packing.

The NPU hashes the channel out of the low address bits and hands the remaining address to the memory controller, whose linear RoBaRaCoCh mapper slices it from the bottom. For this configuration the fields land at fixed positions in the *global* address:

bits [0:5)	byte within the 32 B request
bits [5:9)	channel hashed away, 16 channels
bits [9:14)	column 32 requests × 32 B = one 1 KB row per channel
bits [14:19)	bank slot pch(1) + bankgroup(2) + bank(2) = 32 banks
bits [19:)	row

Two constants follow from that map. **16 KB is 2^{14}** , and a 16 KB contiguous run scatters its 512 requests thirty-two to a channel, where they differ only in the column field — so one such run is one row of one bank *in every channel*. That is what a row-stripe means here. **512 KB is 2^{19}** , thirty-two stripes, one per bank slot, after which the row index increments. Substituting $U = 16$ KB and $NH = 32$ into the formula of §3 gives

$$\text{addr} = \underset{[- \text{ row } -]}{c} \times 2^{19} + \underset{[- \text{ bank } -]}{h} \times 2^{14} + \underset{[- \text{ column } -]}{\text{within}} \quad (\text{within} < 2^{14}, h < 32)$$

which writes the head index into exactly the bits the mapper reads as a bank. Nothing in the controller or the address mapper changes; the placement simply arranges for the head to *be* the bank.

A single element

Head 5, token 100, dimension 0, with sixteen-token blocks, so logical block 6 at offset 4:

p	=	(6 × 16 + 4) × 256 B	=	25,600
c	=	25,600 / 16,384 = 1	within =	9,216
addr	=	1 × 524,288 + 5 × 16,384 + 9,216	=	615,424 = 0x96400
bits [14:19)	=	5	→	bank slot 5 = the head index
bits [19:)	=	1	→	row 1 = the chunk index

Token 164 lands in bank 5, row 2; token 1,562 in bank 5, row 24. Head 6 at the same tokens occupies bank 6 and the same rows. A head walks its own bank one row at a time, and no other head can reach it.

Two benefits, which fail independently

The row is full of one head. All thirty-two request slots of every row visit serve the head that opened it. A 4 KB unit fills eight of thirty-two, so three quarters of the cost of each activation is spent bringing in data for heads that are not being read. This benefit requires only $U \geq 16$ KB, which block size 64 delivers on its own.

The head owns the bank. No other stream can close head 5's row, so a row visit ends when that head is finished with it rather than when a neighbour arrives. This benefit additionally requires the *alignment*: U exactly 2^{14} , and the tensor base padded to a 512 KB boundary so the head bits are not rotated by wherever the allocator placed the tensor. Get U right and the base wrong and the first benefit survives while the second does not, which accounts for most of the gap between block-64's 86.1% row hit and the aligned placement's 91.9%.

The dial

Setting $U = K \times 16$ KB gives each head K banks, with

```
bank = (h×K + c mod K + tile×stride) mod 32
row  = (c / K) × S + h / (32/K)      S = heads sharing a bank set
```

At $K = 1$ a head owns one bank and its consecutive rows are in that same bank, so the stream must close each row before opening the next and cannot overlap its own switch. At $K = 2$ a head alternates between two banks, so the second delivers while the first precharges, at the price that thirty-two heads over sixteen bank pairs means two heads share each pair, separated in the row field. K trades isolation for intra-stream overlap, and §7 measures the trade.

Two further placements follow from the same reasoning and are measured in §6. Keys and values of one head derive from the same offset and therefore share a bank, and the attention tile issues them back to back, so a head's stream contends with itself; displacing the value tensor by sixteen bank slots separates them. And because a 16 KB chunk is four sixteen-token blocks, a fragmenting allocator can split it across rows; running the placement with sixty-four-token blocks makes one physical block exactly one chunk and removes that dependence on allocator behaviour.

The geometry constants are parameters rather than literals. The two numbers the placement needs — the size of a row-stripe and the number of bank slots — are read from the configuration rather than compiled in, because a different DRAM organisation puts the bank field at different address bits. Porting the placement to another memory is a matter of recomputing those two values from its row size, channel count and bank hierarchy; the formula is unchanged.

PART III

Method

5 Model, workload, and what is measured

A cycle-level NPU driving a cycle-level DRAM, with the three streams separated at the controller.

The NPU is ONNXim configured as four 128×128 weight-stationary systolic cores at 1 GHz with a three-deep tile pipeline; the memory system is Ramulator2 with an HBM3 device at 6.4 Gbps, sixteen channels of two pseudochannels $\times$ four bank groups $\times$ four banks, 1 KB rows, 32 B requests, 819.2 GB/s of peak bandwidth. Attention runs head-parallel: head h is assigned to core $h \bmod 4$, and one request's attention completes before the next begins. The model is LLaMA-2 7B, one transformer block, fp16, with a paged KV cache.

The workload is one decode step over **128 requests drawn from the Azure LLM Inference Dataset**, contexts from 169 to 5,305 tokens, 200,064 tokens of cache in total, which is where the 90% KV share comes from. A second, eight-request workload is used where a weight-dominated mix is informative; it moves less, and the ordering of the placements rather than the magnitudes is what transfers.

Block-major here means the arrangement a paged KV cache produces: a page holds B tokens of every head, so within a page one head's keys occupy $B \times 256$ B contiguously and the next head follows immediately. Implementations may subdivide a head's slice further for vectorised access, and may lay keys and values out differently from one another; that finer structure is not modelled here, and it is orthogonal to the page-and-head nesting this report varies. Keys and values are modelled as two tensors with identical internal layout at separate bases.

Separating the streams

Row-buffer statistics are meaningless if the streams are mixed, and they cannot be separated by address range alone: activation tensors occupy the same region as the KV pool and run at about 37% row hit, so averaging them into the KV class understates it by several points. Each memory request therefore carries the operand identifier of the instruction that issued it — query, key, value, weight, output — through the NPU, the interconnect and into the controller, which reports hit, miss and conflict counts for weights, KV and activations separately. The same identifier is written into the address traces, so offline analysis needs no threshold either.

One transport detail affects the measurement enough to be worth naming. The stock model fans a core's single request queue across sixteen injection ports and the interconnect drains a channel's inputs round-robin across those ports, so a core's requests arrive at one channel locally reordered. A bank-hopping stream does not notice; a bank-confined stream has its consecutive rows interleaved and the controller ping-pongs between them. Injecting on the port that serves the request's channel — one queue per core-channel pair, which is what a crossbar gives a single flow — reduced row conflicts by 35 to 45% on every placement, and all results here use it.

Validation

The model is deterministic: identical configurations produce bit-identical cycle counts, so every difference reported is real and the only question is whether it is large. The independent arrival of block-64 and the $K =$

1 placement at the same runtime, from two unrelated arguments, is a useful cross-check on the address generation.

Five address-generation defects were found and fixed in the simulator during this work, and they are worth naming because of what they have in common. Each attention head read KV head 0 rather than its own; long contexts re-read their first chunk once per tile; the KV-concat tiles were marked as skipped, so the cache was never written; a scratchpad bank rotation flushed a bank that a live tile still referenced; and the channel hash exited silently for channel counts other than 16, 32 and 64. **None of these changes the number of addresses issued**, which is what a count-based validation checks, and all of them produce plausible-looking results. The check that catches them is a controlled before-and-after on a fixed workload after any change to address generation.

PART IV

Results

6 The production batch

Row-buffer hit rate does not order the placements by speed, and the reason is legible in the miss and conflict columns.

Table 3 128 Azure requests, one decode step, four cores. KV figures are operand-tagged. The weight stream sits at 74% row hit in every row and is omitted. Cycles are DRAM cycles at 3.2 GHz.

PLACEMENT	KV HIT	MISS	CONFL	KV RD / ACT	ACT / CH	CYCLES	WHILE-BUSY
block-major, 16-token pages	79.4%	4.6%	12.0%	4.9	1.70M	24.13M	68.7%
head-major	70.0%	22.1%	7.9%	3.3	2.31M	17.68M	98.5%
bank-isolated, K = 1	91.9%	3.2%	4.9%	12.5	0.90M	23.21M	71.5%
bank-isolated, K = 2	83.2%	11.9%	4.9%	6.0	1.46M	18.88M	90.2%
K = 2, V displaced, block 64	82.2%	14.6%	3.2%	5.6	1.53M	18.12M	94.4%

Head-major runs the batch 1.36× faster than block-major paging while holding nine points *less* row locality and issuing 36% *more* activations. Each placement has a characteristic failure, and naming them explains the table.

Block-major's high hit rate and its low bandwidth are the same fact. A row holds four heads' 4 KB slices, and with head-parallel assignment the four cores work on heads h through $h+3$ at the same time, so one activation serves four cores at once. That cooperation is where its 79.4% comes from: it is a shared hit rate rather than a private one. But four adjacent heads occupy $4 \times 4 \text{ KB} = 16 \text{ KB}$ contiguous, which is *exactly one bank slot* — so sharing a row means sharing a bank. Counting distinct banks touched per 2,000-cycle window on one channel, block-major reaches a **median of 8 of 32**, the separate key and value tensors and the tiles in flight accounting for the multiple of the single slot the arithmetic predicts. With a quarter of the banks carrying all of the traffic, a row switch has nothing to hide behind: those banks go into precharge and the bus idles. It measures 68.7% while-busy, and its non-hits are the expensive kind, 12.0% conflicts against head-major's 7.9%.

Head-major spreads instead of sharing. Its stream hops to a different bank at every row boundary, so the next row's activate issues on an idle bank while the current row is still delivering, and the same window measurement finds a **median of 27 of 32** banks active. Some bank is always ready, which is the 98.5%. The price is that nothing is shared: it opens the most rows of any placement and hits the fewest of them.

The two metrics answer different questions. A hit rate asks whether the row a request wants was already open; while-busy bandwidth asks whether *any* bank was ready to deliver. Concentration raises the first and destroys the second, which is why the placements do not order the same way under the two.

The isolated placement fails least often — 4.9% conflicts, 3.2% misses, 12.5 KV reads per activation against block-major's 4.9 — and is nevertheless slower than head-major, for the same reason in its sharpest form: at a median of 4 banks active it has the least parallelism of the three, because a head's keys

and values share its bank and a stream confined to one bank cannot overlap its own row switch. At **0.90M activations per channel against 1.70M and 2.31M for the same bytes moved**, it is the placement to choose when the objective is row-buffer energy rather than time.

The last row is the joint operating point: two banks per head, values displaced to a separate bank, sixty-four-token blocks. It holds **82.2% hit, 3.2% conflicts and 94.4% while-busy, 1.33× against block-major paging** — within 2.5% of the fastest placement while keeping twelve points more locality and a third fewer activations than it.

7 Concurrency, and the isolation dial

Isolation requires that concurrently streaming heads not outnumber banks.

Concurrent streams are cores times tile-pipeline depth — twelve at four cores, forty-eight at sixteen — against thirty-two bank slots per channel. The dial and the core count therefore interact, and the interaction has a threshold.

Table 4 KV row hit against concurrency. The eight-request rows are a weight-dominated workload and move less; the ordering is what transfers.

CONFIGURATION	STREAMS / BANKS	BLOCK 16	HEAD-MAJOR	K = 1	K = 2
1 core, 8 req	3 / 32	76.7%	78.4%	92.0%	93.8%
4 cores, production batch	12 / 32	79.4%	70.0%	91.9%	83.2%
16 cores × 32×32	48 / 32	68.8%	74.1%	66.3%	73.3%

At sixteen cores the placement loses most of its locality — 66% row hit — and we can say what does *not* cause it. Conflicts stay at 5.0%, identical to four cores, so the bank isolation has not broken and heads are not evicting one another; the losses are all misses. Nor is it the array, since a sixteen-core run at the full 128×128 width behaves the same way. What collapses is the run: KV reads per activation fall from 12.5 to 3.0, so each row is opened and reopened many times over. Higher concurrency fragments each stream's row visits, and the mechanism is unresolved. The same signature appears when concurrency is raised without adding cores, by interleaving a core's outstanding loads across tiles: bandwidth rises from 71.5% to 78.5%, reads per activation fall from 12.5 to 5.7, row hit falls from 91.9% to 85.1%, and end-to-end time does not move.

The pattern across all three attempts is consistent enough to state as a finding: **on this machine every mechanism that raises bank-level parallelism also shortens the run a stream pulls from each row**, so bandwidth and locality trade rather than compose. Whether that coupling is fundamental or an artifact of how the attention operator issues its loads is the open question, and the quantity to attack is run length rather than bank count — even the best placement consumes only 12.5 of a row's 32 request slots per visit.

In the other direction, at one core the dial costs almost nothing, and $K = 2$ is the better setting at both ends: with three streams in flight the bank-mate sharing never materialises, while the second bank still buys the overlap.

What isolation costs, and why the controller cannot pay it

A bank-confined stream pays the row cycle in series: activate, deliver 64 cycles of data, close, precharge. That is roughly 117 cycles per 64 cycles of data, about 55% duty on its own bank, and the only thing that can fill the gap is a *different* bank — the stream's own second bank at $K \geq 2$, or another head's.

This is not a scheduling failure, and the experiment that settles it is worth reporting as a negative result. We added a scheduler tier that issues precharge and activate ahead of older ready row hits, on the argument that those commands occupy only the command bus and that the measured 62-cycle precharge-to-activate gap was mostly queueing slack above a 23-cycle floor. It worked exactly as designed: the gap fell to **28.5 cycles**. Runtime at one core changed by **127 cycles out of 7.4 million**, and at four cores the tier was 3.2% *worse*, because preempting row hits to issue activates costs more continuity than it recovers. When every queued request targets a bank in the middle of its own row switch, a scheduler has nothing left to reorder.

The same isolation applied to the weight stream is likewise inert: 73.9% to 73.1% row hit, conflicts 4.6% to 3.5%, runtime within 0.2%. The weight stream's four-core loss is 21.5% misses against only 4.6% conflicts — a stream meeting its own row switches, not eviction by neighbours — and it already serves a full queue at 99.4%, so there is no bandwidth there to recover.

8 What holds, and what is not settled

One statement transfers across every configuration measured. Four do not, and one of them bounds the headline number.

Across one, four and sixteen cores, page sizes from 16 to 256 tokens, a weight-dominated and a KV-dominated workload, and a speculative-decoding workload, one statement holds: **the number of DRAM activations required to read a KV cache is set by how many bytes of a single head stay contiguous, and it is minimised when that unit is one row-stripe**. Activation counts are a property of the address stream and of the row geometry, and nothing in the scheduler or the queueing changed them by more than a few percent in any experiment here.

The runtime figures are scoped to one address mapping. All results use a linear row-bank-column mapper, in which the bank index is a contiguous field of the physical address — the arrangement §4 relies on. Controllers that hash other address bits into the bank field are common, and such a mapper would scramble the head-to-bank assignment this placement constructs, changing both the locality it achieves and the bank parallelism available to the placements it is compared against. We do not have a validated implementation of such a hashing scheme, and the ratios in Table 3 should be read as conditional on a linear mapper rather than as properties of the placements alone. Establishing how each placement behaves under a reference hashing scheme is the single most useful extension of this work.

Four things this model cannot settle, and one inequality that replaces a number.

Energy is reported as a ratio, not a figure. Row-activation counts are measured and differ by more than a factor of two between placements moving identical bytes, and activation energy is a real and substantial share of DRAM energy. But converting that into joules needs per-activation and per-bit constants for this device that we do not have, and the absolute figure would inherit all of their error. The defensible statement is the one the measurement supports directly: for the same traffic, the bank-isolated placement issues 0.90M activations per channel against 1.70M and 2.31M, and whatever a row activation costs on this part, it costs that much less often.

Attention concurrency is modelled differently than a GPU serving stack. This simulator runs attention one request at a time, so at four cores the twelve concurrent streams are twelve heads of a single request. A paged-attention kernel on a GPU processes many requests concurrently across streaming multiprocessors, which produces a different stream population over the same banks. Bank isolation is designed against the former; the mapping from streams to banks under the latter is outside what this model can answer, and it is the first thing to check before applying the result to a GPU.

Allocator behaviour is modelled, not observed. A 16 KB chunk spans four sixteen-token blocks, so a fragmenting allocator can split a row-stripe across rows and erode the first of the two benefits in §4. Running at block 64 removes the dependence entirely, which is the practical reason to prefer it over the aligned placement at block 16.

Two speculative-decoding observations are weakly supported. Placement is nearly irrelevant there — head-major and $K = 2$ finish within 0.6% — but KV is only 7% of that workload's traffic in the configuration measured, and the write-after-write traffic that speculative rollback produces has not been characterised in its own right.

Reproduction

The placements are runtime parameters rather than builds. The KV layout, the banks-per-head dial, the value displacement, the bank-chunk and bank-count geometry, the per-channel injection ordering and the stream tagging are all environment variables defaulting to stock behaviour, so an unmodified run reproduces block-major paging exactly. Cycle-stamped DRAM address traces for the three principal placements — 122,117,889 requests each, identical bytes, carrying arrival cycle, decoded bank and row, and the operand tag — are retained alongside the simulator logs.